

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Duration :60 Mins
Off Class
Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

I E.Kavya(192424365) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

E.Kavya

Signature of the student:

Annexure A

DESIGN TASK

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	10	Well-structured documentation with diagrams, steps, and	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation

		clarity			
--	--	---------	--	--	--



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Name:E.Kavya

Reg NO:192424365

DESIGN TASK

1.Design a parser to recognize while loop statements.

Grammar

$S \rightarrow \text{while} (E) \text{Block}$

$E \rightarrow \text{id} \ \&\gt; \ \text{id}$

$\text{Block} \rightarrow \{ \text{Statement} \}$

$\text{Statement} \rightarrow \text{id} = \text{id} ;$

Input

`while(a>b)`

`{`

`x=y;`

`}`

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the loop statement.

(iv) Display Accepted or Rejected.

(v) Handle syntax errors.

Aim

To design and implement a parser to recognize and validate while loop statements using ANTLR by creating lexer and parser rules, and to display whether the input is Accepted or Rejected with appropriate syntax error handling.

Procedure

1. Open the ANTLR Lab (<https://labantlr.org>).
2. Create a grammar file containing the lexer and parser rules for the given while loop grammar.
3. Define lexer rules to recognize keywords, identifiers, operators, parentheses, braces, assignment operator, and semicolon.
4. Define parser rules for the while statement, expression, block, and statement.
5. Enter the input while loop statement in the input window.
6. Set the start rule as program.
7. Click Run to execute the parser.
8. Observe the parse tree and parser console.
9. If the input follows the grammar, the statement is Accepted; otherwise, syntax errors are displayed and the statement is Rejected.

Output:

Not secure lab.antlr.org

Lexer Parser

loop

```

1 grammar WhileLoop;
2
3 program : whileStmt EOF ;
4
5 whileStmt
6 : WHILE LPAREN expr RPAREN block
7 ;
8
9 expr
10 : ID GT ID
11 ;
12
13 block
14 : LBACE statement RBACE
15 ;
16
17 statement
18 : ID ASSIGN ID SEMI
19 ;
20
21 WHILE : 'while';
22 LPAREN : '(';
23 RPAREN : ')';
24 LBACE : '{';
25 RBACE : '}';
26 ASSIGN : '=';
27 GT : '>';
28 SEMI : ';';
29
30 ID : [a-zA-Z]+;
31
32 WS : [ \t\r\n]+ -> skip;

```

Input

example1.txt

```

1 while(a>b)
2 {
3   x=y;
4 }

```

Start rule

program

Run

Show profiler

Tree Hierarchy

```

graph TD
    program1[program:1] --> whileStmt1[whileStmt:1]
    program1 --> EOF[<EOF>]
    whileStmt1 --> while[while]
    whileStmt1 --> expr1[expr:1]
    whileStmt1 --> block1[block:1]
    expr1 --> a[a]
    expr1 --> gt[>]
    expr1 --> b[b]
    block1 --> statement1[statement:1]
    statement1 --> x[x]
    statement1 --> eq[=]
    statement1 --> y[y]
    statement1 --> semicolon[;]

```

Result

The parser for recognizing while loop statements was successfully implemented using ANTLR. The lexer correctly identified the tokens, and the parser validated the input according to the specified grammar. Valid while loop statements were accepted, while invalid statements generated appropriate syntax error messages, satisfying the required design objectives.

2.Design a parser to recognize for loop statements.

Grammar

$S \rightarrow \text{for} (A ; C ; I) \text{Block}$

$A \rightarrow \text{id} = \text{num}$

$C \rightarrow \text{id} \< \text{num}$

$I \rightarrow \text{id} ++$

$\text{Block} \rightarrow \{ \}$

Input

```

for(i=0;i<10;i++)
{
}

```

Design Requirements

- Implement the lexical analyzer using Lex.
- Implement the parser.

- (iii) Validate the loop statement.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Aim

To design and implement a parser to recognize and validate for loop statements using ANTLR by creating lexer and parser rules, and to display whether the input is Accepted or Rejected with syntax error handling.

Procedure

1. Open the ANTLR Lab (<https://labantlr.org>).
2. Create a grammar for the given for loop syntax.
3. Define lexer rules for keywords, identifiers, numbers, operators, parentheses, braces, and semicolons.
4. Define parser rules for the assignment, condition, increment, and block.
5. Enter the input for loop statement.
6. Set the Start Rule as program.
7. Click Run.
8. If the parse tree is generated successfully, the input is accepted.
9. If the input contains syntax errors, ANTLR displays an error message and the input is rejected.

Output:

The screenshot shows the ANTLR Lab web interface at labantlr.org. The browser address bar shows "Not secure labantlr.org". The interface is divided into several sections:

- Lexer Parser:** A dropdown menu is set to "loop". The grammar rules are displayed on the left:


```

1 grammar ForLoop;
2
3 program : forStmt EOF ;
4
5 forStmt
6   : 'for' '(' assign ';' condition ';' increment ')'
7   ;
8
9 assign
10  : ID '=' NUM
11  ;
12
13 condition
14  : ID '<' NUM
15  ;
16
17 increment
18  : ID '++'
19  ;
20
21 block
22  : '{' '}'
23  ;
24
25 ID : [a-zA-Z]+ ;
26 NUM : [0-9]+ ;
27
28 WS : [\t\r\n]+ -> skip ;
      
```
- Input:** The input text is "for(i=0;i<10;i++) { }".
- Start rule:** The start rule is set to "program".
- Run:** A green "Run" button is visible, along with a "Hide profiler" button.
- Tree Hierarchy:** A parse tree is displayed, showing the hierarchical structure of the input:


```

graph TD
    program1[program:1] --> forStmt1[forStmt:1]
    program1 --> EOF1[<EOF>]
    forStmt1 --> for1[for]
    for1 --> LP1["("]
    for1 --> assign1[assign:1]
    for1 --> sem1[";"]
    for1 --> condition1[condition:1]
    for1 --> sem2[";"]
    for1 --> increment1[increment:1]
    for1 --> RP1[")"]
    for1 --> block1[block:1]
    assign1 --> i1[i]
    assign1 --> eq1["="]
    assign1 --> 01[0]
    condition1 --> i2[i]
    condition1 --> lt1["<"]
    condition1 --> 10[10]
    increment1 --> i3[i]
    increment1 --> plus1["++"]
    block1 --> L1["{"]
    block1 --> R1["}"]
      
```
- Parser profile info:** A table with columns: Rule, Invocations, Time (ms), Total k, Max k, and Ambiguities DFA cache miss.

Result

The parser for recognizing for loop statements was successfully implemented using ANTLR. The lexer identified the required tokens, and the parser validated the input according to the specified grammar. Valid for loop statements were accepted, while invalid statements generated appropriate syntax error messages, thereby satisfying all the required design objectives.

3.Design a parser to recognize switch-case statements.

Grammar

$S \rightarrow \text{switch (id) \{ CaseList \}}$

$\text{CaseList} \rightarrow \text{case num : Statement break ; CaseList}$

$\text{CaseList} \rightarrow \text{default : Statement}$

$\text{Statement} \rightarrow \text{id = num ;}$

Input

switch(x)

{

case 1: y=10; break;

default: y=0;

}

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the switch statement.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Aim

To design and implement a parser to recognize and validate switch-case statements using ANTLR by creating lexer and parser rules, and to display whether the input is Accepted or Rejected with syntax error handling.

Procedure

1. Open the ANTLR Lab (<https://labantlr.org>).
2. Create a grammar for the given switch-case statement.

3. Define lexer rules for keywords, identifiers, numbers, operators, braces, parentheses, colon, and semicolon.
4. Define parser rules for the switch statement, case list, and assignment statement.
5. Enter the switch-case input in the input window.
6. Set the Start Rule as program.
7. Click Run.
8. If the parse tree is generated successfully, the statement is Accepted.
9. If the input contains syntax errors, ANTLR displays an error message and the statement is Rejected.

Output:

The screenshot shows the ANTLR web interface with the following components:

- Grammar Editor:** Contains the grammar rules for the switch-case statement.
- Input:** The input text is:


```
1 switch(x)
2 {
3 case 1: y=10; break;
4 default: y=0;
5 }
```
- Start rule:** Set to 'program'.
- Buttons:** 'Run' and 'Hide profiler' are visible.
- Tree Hierarchy:** A parse tree is displayed, showing the hierarchical structure of the input. The root is 'program:1', which branches into 'switchStmt:1 <EOF>'. This further branches into 'switch (x) { caseList:1 }', which then branches into 'case 1 : statement:1 break ; caseList:2'. The 'statement:1' for case 1 is 'y = 10 ;', and the 'statement:1' for the default case is 'y = 0 ;'.
- Parser profile info:** A table showing performance metrics:

Rule	Invocations	Time (ms)	Total k	Max k	Ambiguities	DFA cache miss
caseList:0	2	0.012509	2	1	0	2

Result

The parser for recognizing switch-case statements was successfully implemented using ANTLR. The lexer correctly identified the tokens, and the parser validated the input according to the specified grammar. Valid switch-case statements were accepted, while invalid statements generated appropriate syntax error messages, satisfying all the required design objectives.

4.Design a parser to recognize one-dimensional array declarations.

Grammar

$D \rightarrow \text{Type id [num] ;}$

$\text{Type} \rightarrow \text{int} \mid \text{float} \mid \text{char}$

Input

int marks[50];

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the array declaration.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

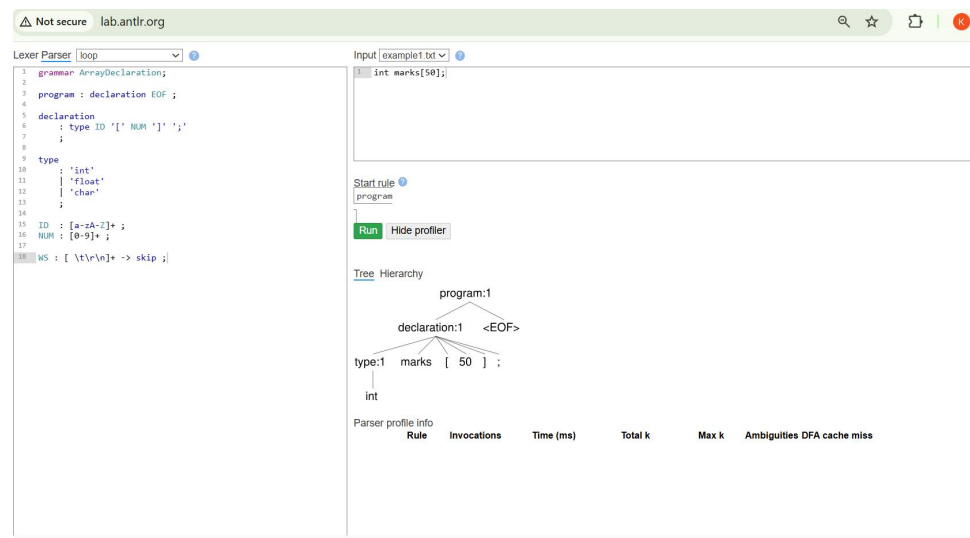
Aim

To design and implement a parser to recognize and validate one-dimensional array declarations using ANTLR by creating lexer and parser rules, and to display whether the input is Accepted or Rejected with syntax error handling.

Procedure

1. Open the ANTLR Lab (<https://labantlr.org>).
2. Create a grammar for one-dimensional array declaration.
3. Define lexer rules for data types, identifiers, numbers, brackets, and semicolon.
4. Define parser rules for the array declaration.
5. Enter the array declaration as input.
6. Set the Start Rule as program.
7. Click Run.
8. If the parse tree is generated successfully, the declaration is Accepted.
9. If the input contains syntax errors, ANTLR displays an error message and the declaration is Rejected.

Output:



Result

The parser for recognizing one-dimensional array declarations was successfully implemented using ANTLR. The lexer correctly identified the required tokens, and the parser validated the declaration according to the specified grammar. Valid array declarations were accepted, while invalid declarations generated appropriate syntax error messages, satisfying all the required design objectives.

5.Design a parser to recognize a simple C program.

Grammar

Program $\rightarrow$ main () Block

Block $\rightarrow$ { Decl Stmt }

Decl $\rightarrow$ int id ;

Stmt $\rightarrow$ id = num ;

Input

main()

{

int a;

a=10;

}

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the program structure.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Aim

To design and implement a parser to recognize and validate a simple C program using ANTLR by creating lexer and parser rules, and to display whether the input is Accepted or Rejected with syntax error handling.

Procedure

1. Open the ANTLR Lab (<https://labantlr.org>).
2. Create a grammar for the given simple C program.
3. Define lexer rules for keywords, identifiers, numbers, braces, parentheses, assignment operator, and semicolon.
4. Define parser rules for the main function, block, declaration, and assignment statement.
5. Enter the given C program as input.
6. Set the Start Rule as program.
7. Click Run.
8. If the parse tree is generated successfully, the program is Accepted.
9. If the input contains syntax errors, ANTLR displays an error message and the program is Rejected.

Output:

The screenshot shows the ANTLR Lab web interface. On the left, the 'Lexer Parser' tab is active, displaying a grammar for a simple C program. The grammar rules are:

```
grammar SimpleCProgram;
program : mainProgram EOF ;
mainProgram : 'main' '(' ')' block ;
block : '{' declaration statement '}' ;
declaration : 'int' ID ';' ;
statement : ID '=' NUM ';' ;
ID : [a-zA-Z]* ;
NUM : [0-9]+ ;
WS : [ \t\r\n]* -> skip ;
```

On the right, the 'Input' tab is active, showing the input code:

```
main()
{
  int a;
  a=10;
}
```

The 'Start rule' is set to 'program'. The 'Run' button has been clicked, and the 'Tree Hierarchy' is displayed. The parse tree structure is as follows:

```
graph TD
    program1[program:1] --> mainProgram1[mainProgram:1 <EOF>]
    mainProgram1 --> main["main ( )"]
    main --> block1[block:1]
    block1 --> declaration1[declaration:1]
    block1 --> statement1[statement:1]
    declaration1 --> int_a[int a ;]
    statement1 --> a_10[a = 10 ;]
```

At the bottom, there is a 'Parser profile info' table with columns: Rule, Invocations, Time (ms), Total k, Max k, and Ambiguities DFA cache miss.

Result

The parser for recognizing a simple C program was successfully implemented using ANTLR. The lexer correctly identified the required tokens, and the parser validated the program structure according to the specified grammar. Valid programs were accepted, while invalid programs generated appropriate syntax error messages, satisfying all the required design objectives.